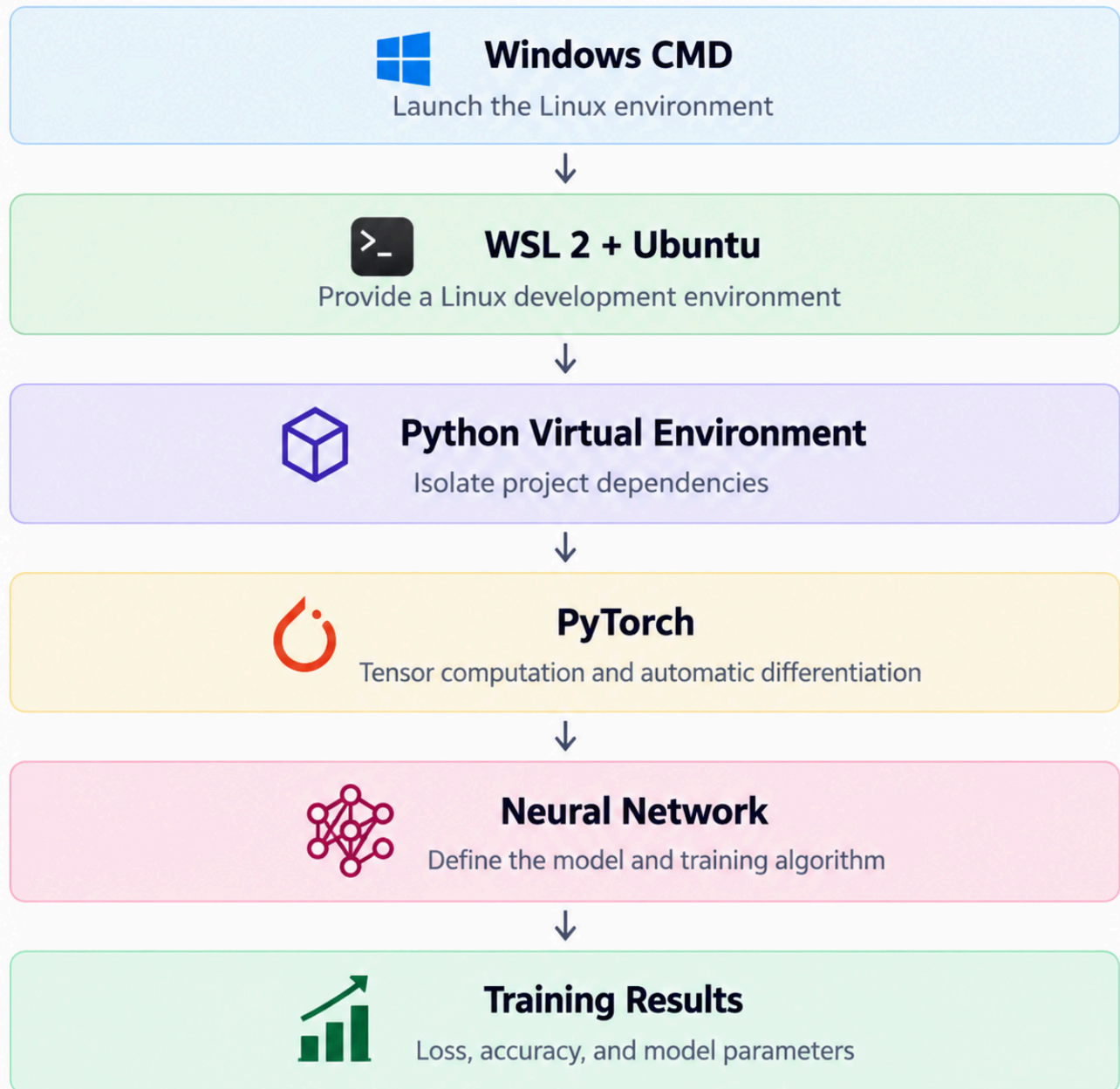


Training Deep Neural Networks

Train a Deep Neural Network from Zero

Overall Workflow



1. Launch Ubuntu from Windows CMD

Press `Win + R`, type `cmd`, and press Enter.

Check WSL:

```
wsl -l -v
```

Launch Ubuntu:

```
wsl -d Ubuntu
```

Navigate to the home directory:

```
cd ~
```

2. Create a Research Project

```
mkdir -p research/first-neural-network  
cd research/first-neural-network  
pwd
```

Expected path:

```
/home/Username/research/first-neural-network
```

3. Set Up Python

Check Python:

```
python3 --version
```

Install the required tools:

```
sudo apt update  
sudo apt install -y python3-venv python3-pip
```

Create a virtual environment:

```
python3 -m venv .venv
```

Activate it:

```
source .venv/bin/activate
```

Your terminal should now display:

```
(.venv) Username@DeviceName:~/research/first-neural-network$
```

4. Install and Verify PyTorch

Install PyTorch and NumPy:

```
python -m pip install torch numpy
```

Check the PyTorch version:

```
python -c "import torch; print(torch.__version__)"
```

Test tensor computation:

```
python -c "import torch; print(torch.rand(2, 2))"
```

A randomly generated tensor confirms that PyTorch is working.

5. Create the Training Script

Open the Nano text editor:

```
nano train.py
```

Paste the following code:

```
import torch
import torch.nn as nn

torch.manual_seed(42)

# 1. Training data
X = torch.tensor([
    [0., 0.],
    [0., 1.],
    [1., 0.],
    [1., 1.]
])

y = torch.tensor([[0.], [1.], [1.], [0.]])
```

```

# 2. Define the neural network
model = nn.Sequential(
    nn.Linear(2, 4),
    nn.Tanh(),
    nn.Linear(4, 1)
)

# 3. Loss function and optimizer
criterion = nn.BCEWithLogitsLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.05)

# 4. Training loop
for epoch in range(2001):
    optimizer.zero_grad()

    logits = model(X)
    loss = criterion(logits, y)

    loss.backward()
    optimizer.step()

    if epoch % 200 == 0:
        print(f"Epoch {epoch}, Loss: {loss.item():.6f}")

# 5. Evaluate the model
with torch.no_grad():
    predictions = torch.sigmoid(model(X))
    print("\nPredictions:")
    print(predictions)

# 6. Save trained weights
torch.save(model.state_dict(), "xor_model.pth")

```

Save the script:

1. Press `Ctrl + O`.
2. Press `Enter`.
3. Press `Ctrl + X`.

6. Execute the Training

Run:

```
python train.py
```

The terminal will display training progress:

```
Epoch 0, Loss: ...  
Epoch 200, Loss: ...  
Epoch 400, Loss: ...  
...  
Epoch 2000, Loss: ...  
  
Predictions:  
tensor(...)
```

Check that the loss decreases and the predictions are close to the expected XOR outputs.

7. Check the Saved Model

List the project files:

```
ls -lh
```

Expected files:

```
train.py  
xor_model.pth
```

The `.pth` file contains the trained model parameters.

8. Restart the Environment Later

After closing CMD, reopen it and run:

```
wsl -d Ubuntu
```

Then:

```
cd ~/research/first-neural-network  
source .venv/bin/activate  
python train.py
```

There is no need to reinstall Python or PyTorch.

9. Workflow Summary

Windows CMD
↓
WSL 2 + Ubuntu
↓
Research Project Directory
↓
Python Virtual Environment
↓
Install PyTorch
↓
Create train.py
↓
Execute Training
↓
Verify Output
↓
Save Model Weights

10. Troubleshooting

Problem	Solution
torch: command not found	Run <code>python train.py</code> , not <code>torch train.py</code>
No module named 'torch'	Activate <code>.venv</code> and install PyTorch
No module named 'numpy'	Run <code>python -m pip install numpy</code>
train.py produces no output	Check whether the file contains code
CMD accidentally closed	Reopen CMD, launch Ubuntu and reactivate <code>.venv</code>
Docker is stopped	Docker is not required for this experiment

Outcome: A working local PyTorch training workflow with a saved neural network model.